



Nombre:

Grupo:

Pregunta 1: Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

[] En el siguiente código C++ se produce una fuga de memoria o *memory leak*:

```
int *k = new int[100];
for (i = 0; i <= 100; ++i) { k[i] = 0; }
...
delete[] k;
```

[] Las implementación normal de un vector dinámico implementa una reducción del tamaño físico *tamf* a la mitad cuando el tamaño lógico *taml* cae por debajo de *tamf/2*.

[] Una clase que implementa una lista circular no incluye un puntero al nodo cabecera de la misma, al contrario que las listas simple y doblemente enlazadas.

[] Trasferir todos los nodos de una lista simplemente enlazada *l1* al final de otra lista enlazada *l2* requiere tiempo O(n) (nota: *l1* queda vacía tras esta operación).

[] Un árbol binario es un grafo normalmente dirigido, conexo y libre de ciclos, donde el grado de salida de todos los nodos es igual o inferior a 2.

[] El cambio de posición de los nodos en una rotación simple de un árbol AVL se implementa con únicamente 4

asignaciones de punteros.

[] En dispersión abierta puede ocurrir que un dato no quede guardado en la posición indicada inicialmente por la función de dispersión.

[] Sea *Complejo* una clase que implementa número complejos de la forma $(a + ib)$ tal como se muestra a continuación. La declaración de un conjunto de números complejos mediante la declaración STL *set<Complejo> complejos;* es correcta.

```
class Complejo {
public:
    float real, imag;

    Complejo(float real, float imag):
        real(real), imag(imag) {}
};
```

[] El borrado lógico de un registro en un fichero de datos puede hacerse con 3 accesos si hay que mantener una lista de registros borrados o únicamente 1 si no existe dicha lista.

[] La profundidad de todas las hojas de un árbol B es la misma.

Pregunta 2:

Insertar (I) o borrar (B) la siguiente secuencia numérica en una tabla hash cerrada de tamaño 5 con cubetas de tamaño 2. La dispersión es lineal con la función $h(x)=x \% tama$. Se pueden unificar varios pasos en uno pero quedando claro en todo momento los estados: vacío, ocupado y disponible.

I(10), I(6), I(2), I(15), I(8), I(20), I(11), B(6), I(7), I(25), B(8), B(10), I(12).

Pregunta 3:

a) Dada la siguiente implementación de un árbol genérico de orden *n*, implementar la operación *int numDescendientes(Nodo<T>* nodo)*, que devuelve el número total de descendientes de un nodo dado (no sólo descendientes directos). Añadir operaciones auxiliares si es necesario.

```
template<typename T>
class Nodo {
public:
    Nodo<T>** hijos;
    T dato;

    Nodo(int orden, const T& dato): dato(dato) {
        hijos = new Nodo<T>*[orden];
        for (int h = 0; h < orden; ++h) hijos[h] = 0;
    }
};

template<typename T>
class Arbol {
    int orden;
    Nodo<T>* raiz;

public:
    Arbol(int orden): orden(orden) {
```

```
    raiz = 0;
}
...
};
```

b) Implementar las operaciones *void Matriz2D<T>::anadirFila()* y *void Matriz2D<T>::anadirColumna()* de la siguiente clase que implementa una matriz bidimensional:

```
template<typename T>
class Matriz2D {
    int nfilas, ncolumnas;
    T** datos;

public:
    Matriz2D(int nfilas, int ncolumnas):
        nfilas(nfilas), columnas(ncolumnas) {

        datos = new T*[nfilas];
        for (int f = 0; f < nfilas; ++f) {
            datos[f] = new T[ncolumnas];
        }
        ...
    };
```

Pregunta 4:

Una empresa que gestiona grandes aparcamientos ha realizado el siguiente diseño con carácter genérico para el control de parkings de aeropuertos, zonas comerciales, etc. El parking puede tener *nplantas* en forma cuadrangular. Todas las plantas son iguales y numeran igual sus plazas de aparcamiento. La numeración va primero por planta, segundo por filas identificadas mediante letras a partir de la A (en sentido de menor a mayor distancia respecto a la entrada/salida), y finalmente las columnas por número. Así, la plaza de aparcamiento 2A5, indica que se encuentra en la segunda planta, primera fila y 5º aparcamiento.

El aparcamiento está dotado de sensores que indican en todo momento con luz verde que el aparcamiento está libre y con rojo que está ocupado. Además, al comienzo y al final de cada fila una cartelería luminosa en forma de doble flecha indica el número de plazas libres si se gira hacia la izquierda (lateral) y si sigue de frente (frente). En el primer caso se indica el número de plazas libres en esa fila, mientras que la de frente indica todas las plazas libres si sigue de frente, es decir, todas las de las filas sucesivas hasta el final del parking. De forma similar, al final de cada fila hay otro indicador que hace lo mismo pero en sentido contrario. En la figura ejemplo aparece la disposición de dicha cartelería (flechas rojas) y el sentido de la circulación (flechas azules).

Como el parking también es usado por empresas de realquiler que facilitan al usuario el aparcamiento y retirada del coche, es muy habitual que el usuario/empresa localice el aparcamiento donde está estacionado un vehículo dando su matrícula.

Las clases tienen la siguiente funcionalidad:

PlazaParking: Esta clase identifica de forma única cada plaza de aparcamiento (nplaza) con los tres caracteres indicados anteriormente. Tiene sensores que se activan automáticamente y llaman a entra *Vehiculo()* cuando detectan que un vehículo entra en el aparcamiento y *saleVehiculo()* cuando se va. En el primer caso un cámara se activa y registra la matrícula del coche. En ambos casos se avisa convenientemente al control del parking con *ParkingControl::entradaVehiculo()* y *ParkingControl::salidaVehiculo()*.

CartelInformativo: Se colocan estos carteles por parejas al comienzo y al final de cada fila. Los vehículos siempre se mueven por el parking en este sentido circular rodeando al parking (puede que haya pasillos centrales pero ahí no se localizan). Los carteles indican las plazas libres, uno de ellos en esa fila si el vehículo gira a la izquierda y otro si se sigue de frente. En este último caso debe contabilizar la suma total de los aparcamientos si sigue de frente hasta el final del aparcamiento (no se tienen en cuenta el resto de plantas). Estos carteles deben estar permanentemente actualizados.

ParkingControl: Gestiona todas las plazas de aparcamiento, así como los carteles informativos. Las entradas y salidas de vehículos se traducen en las correspondientes actualizaciones de plazas libres avisadas en los carteles informativos (pueden verse afectados todos los de una planta), así como el control de los coches que están actualmente estacionados en el aparcamiento (se pueden consultar con *buscaVehiculo()*)

